

Building a Bilingual Legal RAG Chatbot

PPR-Bot — a from-scratch Retrieval-Augmented Generation system over the Bangladesh Public Procurement Rules 2025

A build log & teaching companion: the architecture, the RAG techniques, and every real-world challenge hit along the way.

Prepared as a reference for a video walkthrough on making a RAG system.

1. What we built, in one paragraph

PPR-Bot is a chat assistant that answers questions — in Bangla or English — about a 271-page Bangladeshi government legal document (the Public Procurement Rules 2025, published in the official Gazette). It is a **Retrieval-Augmented Generation (RAG)** system: instead of trusting a language model to remember the law, we retrieve the exact relevant passages from the document and force the model to answer *only* from them, citing the rule number and page. It is hand-written end-to-end — no LangChain or LlamaIndex — specifically so every step is visible and teachable. It is served by a FastAPI backend that streams answers to a KakaoTalk-style chat UI.

The single most important idea to convey in your video: **RAG is not one model call. It is a pipeline.** Most of the engineering — and almost all of the hard problems — live in getting clean text in, chunking it sensibly, indexing it well, and retrieving the right pieces. The language model is the last and easiest 5%.

2. The mental model: why RAG, and what the pieces are

A plain chatbot answers from the model's frozen training memory. For a specific legal document that may be newer than the model — and where a wrong answer is dangerous — that is unacceptable. RAG fixes this by splitting the job into two halves:

- **Offline (build the knowledge base):** turn the PDF into clean text, cut it into searchable chunks, and build search indexes over those chunks. Done once.
- **Online (answer a question):** retrieve the most relevant chunks for the user's question, paste them into a prompt as context, and have the model answer strictly from that context.

Separating these two is itself a core teaching point: the offline side is a set of resumable batch scripts that each produce an artifact on disk; the online side is a fast server that only reads those artifacts. They never get tangled together.

3. Architecture & data flow

Offline pipeline (run once, in order)

- **Extraction** — render each PDF page to an image and have a vision LLM transcribe it to clean Markdown (this dodges the document's broken fonts; see Challenge 1).
- **Chunking** — parse the Markdown into the legal hierarchy (Part > Chapter > Rule > Sub-rule) and cut one chunk per rule/section, keeping a breadcrumb and source page.
- **Enrichment** — for each chunk, an LLM writes a 1–2 sentence 'situating' summary that is prepended before indexing (Anthropic's Contextual Retrieval).
- **Indexing** — embed every chunk into a dense vector (bge-m3) AND build a keyword index (BM25). Two complementary indexes.

Online serving (per question)

- **Condense** — rewrite a conversational follow-up into a standalone query using chat history.
- **Hybrid retrieve** — run dense (semantic) and sparse (keyword) search, fuse the two ranked lists with Reciprocal Rank Fusion (RRF).
- **Rerank** — a cross-encoder re-scores the top candidates for precision (now toggleable).
- **Generate** — stream a grounded, cited answer from the retrieved context.
- **Suggest** — propose follow-up questions for the UI.

Tech stack: FastAPI + Uvicorn + SSE streaming; Google Gemini for OCR / generation / query rewriting; BAAI/bge-m3 embeddings and BAAI/bge-reranker-v2-m3 cross-encoder (local, CPU); rank_bm25 for keyword search; a hand-rolled NumPy vector store; vanilla HTML/CSS/JS frontend.

4. The RAG techniques used (and why)

This system deliberately layers several 'beyond-basic' RAG techniques. For the video, each one is a great mini-segment: show the naive version, show why it fails, then add the technique.

4.1 Vision-LLM OCR ingestion

Rather than extracting the PDF's text layer (which was garbled), we render each page to an image and ask a vision model to transcribe it. This treats every page — clean text, broken-font text, or scanned image — identically.

4.2 Structure-aware chunking

Naive RAG chops text into fixed 500-character windows, which slices rules in half. We instead chunk along the document's own hierarchy, so each chunk is a coherent rule and carries its breadcrumb (Part > Chapter > Rule) and source page as metadata for citation.

4.3 Contextual Retrieval (Anthropic's technique)

A lone chunk like 'the limit shall not exceed 3 lakh taka' is meaningless to a search engine — which rule? about what? Before indexing, an LLM writes a short blurb situating the chunk ('This is from Rule 76 on direct procurement, stating the goods threshold...') and we prepend it. Both indexes then capture context the raw chunk lacked. This measurably reduces 'relevant but unfindable' misses.

4.4 Hybrid search = dense + sparse

Dense (embeddings) captures meaning — a query about 'purchase limits' matches a chunk about 'procurement thresholds' even with no shared words. **Sparse** (BM25) captures exact terms — rule numbers, specific legal phrases — which embeddings can blur. Real questions need both, so we run both.

4.5 Reciprocal Rank Fusion (RRF)

How do you merge two ranked lists whose scores are on totally different scales (cosine similarity vs BM25)? RRF ignores the raw scores and uses

only the *rank position*: each item scores $1/(k + \text{rank})$ in each list, summed. Simple, robust, no tuning. A perfect small teaching unit (it fits in ten lines and is easy to unit-test).

4.6 Cross-encoder reranking

Hybrid search is fast but approximate — it compares query and chunk vectors that were computed separately. A cross-encoder reads the query AND a chunk *together* and outputs one relevance score: far more accurate, far slower. So we only run it on the ~10–15 candidates hybrid search already shortlisted. This 'retrieve cheap, rerank precise' two-stage pattern is standard in modern RAG.

4.7 Conversational query condensation

Follow-ups like 'what about for works?' are meaningless to a search engine. Before retrieving, a cheap LLM call rewrites the follow-up into a standalone query using recent turns. This is what makes multi-turn RAG actually work.

4.8 Grounded generation with citations

The system prompt forces the model to answer only from the provided context, cite the rule and page for each fact, and explicitly say when the answer is not present — the main defence against hallucination. We observed this working: asked about exact thresholds not yet in the extracted pages, the bot correctly said the figures were not in its context instead of inventing them.

4.9 Follow-up suggestions

After answering, a cheap LLM call proposes 3 natural next questions, returned to the UI as tappable chips. Generating these on the server (not hardcoding them) keeps them relevant to the actual conversation.

4.10 A latency/precision toggle (a RAG variation worth discussing)

On a slow CPU the cross-encoder rerank dominated response time. We added a per-request toggle: with rerank ON you get maximum precision (~30s here); with it OFF you fall back to hybrid+RRF ordering only (~5s),

retrieving the *same* content with slightly less precise ordering. This is a real engineering dial every RAG builder should understand: **retrieval quality vs latency vs cost**.

5. The challenges — the heart of the story

This is the most valuable part for a video: real problems, in the order they bit us, each with its symptom, root cause, and fix. Many of these are not in tutorials because tutorials use clean data and unlimited APIs.

1. The PDF text was unreadable mojibake

Symptom	Extracting the PDF's text layer produced garbage like 'evsjvž`k' instead of Bangla.
Root cause	The document uses a legacy non-Unicode Bangla font (SutonnyMJ) that maps Bangla glyphs onto Latin code points. Direct text extraction reads the code points, not the glyphs.
Fix	Stop extracting text; render each page to an image and have a vision LLM transcribe what it sees into proper Unicode. Font-agnostic, and it also handles scanned pages.
Lesson	Inspect your raw data before building anything. The hardest RAG problems are data problems.

2. Free-tier API quota kept running out mid-job

Symptom	The 271-page extraction halted partway, every further call returning 429 RESOURCE_EXHAUSTED.
Root cause	The free Gemini tier caps each model at ~20 requests/day. A 271-page OCR job plus ~170 enrichment calls vastly exceeds that.

Fix	Two fixes: (a) make every batch job resumable — write each page to disk with a status manifest and skip completed work on re-run; (b) use per-step fallback model chains — each model name is a separate daily bucket, so trying a second model name doubles the allowance. The job now legitimately spans several days.
Lesson	Design long LLM batch jobs to be idempotent and resumable from day one — they <i>will</i> be interrupted.

3. Using the cheap model for OCR garbled the Bangla

Symptom	Switching OCR to a cheaper 'lite' model produced romanized mojibake and repetition loops.
Root cause	Small models can generate fluent text but cannot reliably <i>read</i> dense rendered Bangla script.
Fix	Match the model to the task: full models for vision OCR (quality matters), the cheap lite model only for plain text generation like chunk-summaries (where it is fine and the daily quota is bigger).
Lesson	'Cheaper model' is a per-task decision, not a global one.

4. Some pages were silently saved as empty files

Symptom	About a dozen pages produced 0-byte Markdown files yet were marked 'done'.
Root cause	When the model returned an empty response, the code saved the empty string and recorded success.
Fix	Treat an empty transcription as an error (raise, don't save), and make the resume check require the output file to be non-empty before skipping it.

Lesson	Validate every external response. 'It returned without error' is not the same as 'it returned something usable'.
---------------	--

5. A model fell into an endless repetition loop

Symptom	One page became 130 KB of the same character repeated — a runaway generation.
Root cause	Degenerate decoding: the lite model got stuck repeating a token.
Fix	A guard that rejects suspiciously long output or long single-character runs, so the bad output is never saved and the page is retried.
Lesson	Add cheap sanity checks on generated output; degenerate loops are common and easy to detect.

6. The job ignored the quota signal and burned through retries

Symptom	After quota ran out, the job kept retrying every remaining page for ages instead of stopping.
Root cause	The 429 error text was attached as an exception 'cause' but not included in the message string the outer loop inspected, so the 'stop on quota' check never matched.
Fix	Detect 429/RESOURCE_EXHAUSTED inside the transcribe call and break to the next model immediately (no backoff sleep); include the underlying error text in the raised message so the orchestrator can stop the whole job cleanly.
Lesson	When you catch-and-re-raise, preserve the information downstream code relies on.

7. Requests hung forever on a flaky connection

Symptom	Both background jobs froze for 10+ minutes with zero progress and no error.
Root cause	The Google SDK's HTTP client had no bounded timeout, so a stalled network read blocked indefinitely — indistinguishable from 'still working'.
Fix	A single shared client factory that sets a 60-second request timeout, so a stuck call raises (and is then retried) instead of hanging.
Lesson	Every network client needs an explicit timeout. 'Hung' is worse than 'failed' because you cannot even tell it happened.

8. Gazette page furniture leaked into the chunks

Symptom	Chunks contained repeated running headers, printed page numbers, and a cover-page price stamp mid-text.
Root cause	The OCR faithfully transcribed the boilerplate that the printed Gazette repeats on every page; concatenating pages dropped it into the middle of rule text.
Fix	A small filter in the parser that drops these known furniture lines, with unit tests proving real sub-rule references that look similar are NOT removed.
Lesson	Clean at the seams. Boilerplate that is harmless on one page becomes noise once pages are joined.

9. The model files would not download (stuck at 0 bytes)

Symptom	Downloading the embedder/reranker (~4.6 GB) stalled at 0 bytes across every method, while small files downloaded fine.
----------------	--

Root cause	The model library's modern chunked transfer protocol hung on this network, even though plain authenticated HTTPS range requests to the very same servers worked at ~2 MB/s.
Fix	Bypass the library's downloader entirely: a small script using plain HTTP range requests with resume + retries, saving into a local folder the code loads models from directly. Also skipped the unused ONNX copies to halve the download.
Lesson	When a high-level helper mysteriously fails, drop one layer down and verify the primitive works. An auth token also bypasses anonymous rate-limit throttling.

10. Answers took ~30 seconds on CPU

Symptom	Every reply took about half a minute before streaming.
Root cause	The 568M cross-encoder reranker scores each (query, chunk) pair with a full forward pass; on a GPU-less, modest CPU that is several seconds per pair.
Fix	Four levers: use all CPU cores (the library defaulted to half); cap the rerank token window; shrink the candidate pool; warm the models at server startup so the first question is not slow. Then a user toggle to skip rerank entirely for ~5s answers.
Lesson	Profile before optimizing — we measured each stage and found 90% of the time was one step. Latency, cost, and quality are dials, not constants.

11. The browser showed no answer and no error

Symptom	Typing dots appeared for ~30s, then vanished with a blank chat — no answer, no error.
----------------	---

Root cause	The server streams Server-Sent Events with CRLF (<code>\r\n</code>) line endings, but the browser parser split frames on <code>\n\n</code> , which never matched — so it silently dispatched zero events. (Server-side Python tests passed because their line reader tolerated CRLF, hiding the bug.)
Fix	Make the SSE parser CRLF-tolerant (split on a <code>\r?\n\r?\n</code> pattern).
Lesson	Test the real client, not just a convenient stand-in. The transport format matters down to the byte.

12. Answers showed literal asterisks instead of formatting

Symptom	Bullets rendered as <code>'*</code> and the italic source line showed raw <code>'* ...*'</code> .
Root cause	The frontend formatter only converted <code>**bold**</code> , leaving the model's Markdown bullets and single-asterisk italics untouched.
Fix	A small, safe Markdown renderer: escape HTML first, then render bullets, bold, and italics (bold before italic so the passes do not collide).
Lesson	If the model emits Markdown, the UI must render Markdown — and always escape before injecting HTML.

13. Windows console crashes on Bangla / dashes

Symptom	Scripts and the server crashed with a cp932 codec error when printing Bangla or an em-dash.
Root cause	The default Windows console encoding cannot encode those characters.

Fix	Reconfigure stdout to UTF-8 with error replacement at the top of every entry point.
Lesson	Cross-platform text output needs an explicit UTF-8 setup; do not assume the terminal's default.

6. How to teach this in a video

A suggested narrative arc that keeps viewers engaged by leading with problems, not jargon:

- **Hook:** ask a plain chatbot a specific legal question; watch it hallucinate or refuse. State the goal: trustworthy, cited answers from a real 271-page law.
- **Show the data reality:** open the PDF, copy-paste the 'text', reveal the mojibake. This instantly motivates the vision-OCR approach (Challenge 1).
- **Build the offline pipeline live,** one artifact at a time: pages to Markdown, Markdown to chunks, chunks to indexes. Emphasize 'each step writes a file you can open and inspect'.
- **Teach each retrieval technique by failure:** show keyword-only missing a paraphrase, then embeddings missing an exact rule number, then hybrid+RRF getting both. Then show rerank fixing a borderline ordering.
- **Wire the online side:** condense to retrieve to generate, streaming token by token over SSE.
- **Tell the war stories:** the quota wall, the 0-byte pages, the silent SSE bug, the 30-second wait and the toggle. These are what make the video memorable and honest.
- **End on the dials:** quality vs latency vs cost — the rerank toggle as the concrete example.

Talking points that land well

- 'RAG is a pipeline, not a prompt.' Most effort is data engineering.
- 'Retrieve cheap, rerank precise' — the two-stage retrieval pattern.
- 'Contextual Retrieval' — a few cents of LLM calls at index time buys real recall.
- 'Make batch jobs resumable' — because free tiers and networks will interrupt you.
- 'Test the real client and validate every external response.'

7. Key takeaways

- The model is the easy part; clean ingestion, sensible chunking, and good retrieval are where the quality and the work are.
- Hybrid (dense + sparse) + RRF + cross-encoder rerank is a strong, explainable default stack.
- Contextual Retrieval is a high-leverage, low-code upgrade.
- Engineer for failure: resumable jobs, bounded timeouts, output validation, graceful degradation.
- Expose the quality/latency/cost trade-off as a real control instead of hardcoding it.
- Ground every answer in retrieved context and force citations; let the model say 'not found'.

Generated for the PPR-Bot project. All code referenced is hand-written and available in the project repository; the offline artifacts (extracted pages, chunks, embeddings, models) are regenerated by the pipeline scripts described above.